ChatGPT

EfficientNet Training Workflow (1)

You

Wake_EfficientNet...
Code

explain each code line simply to explain to my sir simply explain all the lines of code line wise

Analyzed

ChatGPT

I checked your uploaded `Wake_EfficientNet_Training(1).ipynb` notebook. It contains 12 cells, including 10 code cells. I'll explain the code cell by cell, in simple language, so you can understand it and explain it confidently to your sir.

Project explanation – EfficientNet-based Sonar Wake Detection

First, understand what this project does

Your project takes:

- 50 Beam PRI CSV files – containing sonar signal data.
- 1 PRI Analysis CSV file – containing the wake-detection information used as training labels.

The notebook then:

1. Reads the files.
2. Matches each Beam file with its PRI number in the Analysis file.
3. Converts each of the 25 I-channel signals into an image.
4. Trains an EfficientNetB0 deep-learning model.
5. Uses the trained model to predict whether each channel contains a wake signal.
6. Saves one output CSV file for each Beam PRI file.

Overall project workflow

Take your curiosity further
Scan to get the ChatGPT app

Input files
50 Beam PRI CSV files + 1 PRI Analysis CSV

↓

Data preprocessing
Extract I-channel signals and convert them into spectrogram images

↓

EfficientNetB0 training
Learn patterns associated with wake and non-wake signals

↓

Prediction
Predict yes/no for each channel

↓

Output
50 CSV files containing Channel and Label

Cell 1 – Importing libraries

Purpose: Import all the Python libraries required for the project.

Code line	Simple explanation
`import os`	Helps Python work with folders and file paths.
`import re`	Helps find patterns in text, such as extracting PRI numbers from filenames.
`import glob`	Helps find multiple CSV files inside a folder.
`import numpy as np`	Imports NumPy for numerical calculations and arrays.
`import pandas as pd`	Imports Pandas to read and process CSV files.

Take your curiosity further
Scan to get the ChatGPT app

`import tensorflow as tf`	Imports TensorFlow, which is used to build and train the deep-learning model.
`from scipy.signal import spectrogram`	Imports the function that converts a signal into a spectrogram.
`from sklearn.model_selection import GroupShuffleSplit`	Imports a group-based data-splitting method. **It is imported but not used in this notebook.**
`from tensorflow.keras.applications import EfficientNetB0`	Imports the EfficientNetB0 neural-network architecture.
`from tensorflow.keras.layers import (`	Starts importing individual neural-network layers.
`Input, GlobalAveragePooling2D, Dropout, Dense`	Imports the input, pooling, dropout and output layers.
`)`	Ends the layer import statement.
`from tensorflow.keras.models import Model`	Imports the function used to create the complete neural-network model.
`print("TensorFlow:", tf.__version__)`	Displays the installed TensorFlow version.
`print("GPU devices:", tf.config.list_physical_devices("GPU"))`	Checks whether a GPU is available.
`print("Environment ready")`	Displays a message that the setup is ready.

How to explain to sir:

"I first imported the required libraries. Pandas and NumPy are used for data handling, SciPy is used for spectrogram generation, and TensorFlow is used to create and train the EfficientNetB0 deep-learning model."

Cell 2 – Setting project folders and reading file locations

Purpose: Tell Python where the input and output folders are located.

Code line	Simple explanation

Take your curiosity further
Scan to get the ChatGPT app

`BASE_DIR = r"D:\Wake_EfficientNet"`	Defines the main project folder. The `r` makes it a raw Windows path.
`BEAM_DIR = os.path.join(BASE_DIR, "Dataset", "Beam")`	Creates the path to the Beam input folder.
`ANALYSIS_DIR = os.path.join(BASE_DIR, "Dataset", "Analysis")`	Creates the path to the Analysis input folder.
`OUTPUT_DIR = os.path.join(BASE_DIR, "Output")`	Defines where prediction CSV files will be saved.
`MODEL_DIR = os.path.join(BASE_DIR, "Training")`	Defines where the trained model will be saved.
`os.makedirs(OUTPUT_DIR, exist_ok=True)`	Creates the Output folder if it does not already exist.
`os.makedirs(MODEL_DIR, exist_ok=True)`	Creates the Training folder if it does not already exist.
`analysis_files = glob.glob(`	Starts searching for files in the Analysis folder.
`os.path.join(ANALYSIS_DIR, "*.csv")`	Searches for all CSV files inside the Analysis folder.
`)`	Completes the search statement.
`beam_files = glob.glob(`	Starts searching for Beam files.
`os.path.join(BEAM_DIR, "*.csv")`	Searches for all CSV files inside the Beam folder.
`)`	Completes the search statement.
`print("Beam files:", len(beam_files))`	Displays how many Beam files were found.
`print("Analysis files:", len(analysis_files))`	Displays how many Analysis files were found.
`assert len(beam_files) == 50, "Expected 50 Beam files"`	Checks that exactly 50 Beam CSV files are available.
`assert len(analysis_files) == 1, "Expected 1 Analysis CSV"`	Checks that exactly one Analysis CSV file is available.

How to explain to sir:

Take your curiosity further
Scan to get the ChatGPT app

"In this step, I configured the project folders. The program automatically reads the 50 Beam CSV files and one Analysis CSV file. It also checks whether the required number of files is available before proceeding."

Cell 3 – Reading and matching PRI numbers

Purpose: Match every Beam file with its corresponding PRI information in the Analysis CSV.

Code line	Simple explanation
`analysis = pd.read_csv(analysis_files[0])`	Reads the Analysis CSV file into a Pandas DataFrame.
`analysis.columns = analysis.columns.str.strip()`	Removes unwanted spaces from column names.
`print("Analysis columns:")`	Prints a heading.
`print(analysis.columns.tolist())`	Displays all column names in the Analysis file.
`required = ["PRI-No", "Wake_Detection"]`	Specifies the two required columns.
`for col in required:`	Checks each required column one by one.
`if col not in analysis.columns:`	Checks whether the column is missing.
`raise ValueError(f"Missing column: {col}")`	Stops the program and reports the missing column.
`def get_pri_number(value):`	Defines a function to extract a PRI number.
`match = re.search(r'(\d+)', str(value))`	Searches for digits in the given value.
`if not match:`	Checks whether any digits were found.
`raise ValueError(...)`	Stops the program if no PRI number is found.
`return int(match.group(1))`	Returns the extracted number as an integer.
`analysis["PRI_No"] = analysis["PRI-No"].apply(get_pri_number)`	Creates a new column containing the numeric PRI values.
`if analysis["PRI_No"].duplicated().any():`	Checks whether any PRI number appears more than once.

Take your curiosity further
Scan to get the ChatGPT app

`raise ValueError("Duplicate PRI numbers found in Analysis CSV")`	Stops if duplicate PRI numbers exist.
`analysis_map = analysis.set_index("PRI_No")["Wake_Detection"].to_dict()`	Creates a dictionary that connects each PRI number with its Wake_Detection value.
`def get_file_pri_number(filename):`	Defines a function to extract a PRI number from a Beam filename.
`match = re.search(r'PRI[_-]?(\d+)', os.path.basename(filename), re.I)`	Finds a PRI number in the filename, ignoring uppercase/lowercase differences.
`if not match:`	Checks whether a PRI number was found.
`raise ValueError(...)`	Stops if the filename does not contain a PRI number.
`return int(match.group(1))`	Returns the PRI number as an integer.
`for path in beam_files:`	Goes through every Beam file.
`pri = get_file_pri_number(path)`	Extracts the PRI number from that file.
`if pri not in analysis_map:`	Checks whether the same PRI number exists in the Analysis dictionary.
`raise ValueError(...)`	Stops if the Beam file has no matching Analysis entry.
`print("All 50 Beam files matched successfully")`	Displays the success message after all matches are checked.

How to explain to sir:

"Here, I read the Analysis CSV and extract the PRI numbers. Then I match each Beam filename with its corresponding PRI number in the Analysis file. This ensures that the correct wake-detection information is associated with each Beam file."

Cell 4 – Decoding the wake-detection mask

Purpose: Convert the hexadecimal wake-detection value into 25 channel labels.



Take your curiosity further

Scan to get the ChatGPT app



Code line	Simple explanation
`BIT_ORDER = "lsb_first"`	Specifies that the least significant bit is assigned to the first channel.
`def decode_mask(mask_value, number_of_channels=25):`	Creates a function to decode the mask into 25 labels.
`mask_text = str(mask_value).strip()`	Converts the mask into text and removes extra spaces.
`mask_number = int(mask_text, 0)`	Converts the hexadecimal or other prefixed numeric text into an integer.
`labels = []`	Creates an empty list to store channel labels.
`for channel_index in range(number_of_channels):`	Repeats the operation for all 25 channels.
`if BIT_ORDER == "lsb_first":`	Checks whether the first channel starts from the least significant bit.
`bit_position = channel_index`	Uses the current channel index as the bit position.
`else:`	Runs if the bit order is different.
`bit_position = number_of_channels - 1 - channel_index`	Calculates the reverse bit position.
`bit_value = (mask_number >> bit_position) & 1`	Extracts one bit from the mask.
`labels.append("yes" if bit_value == 1 else "no")`	Stores "yes" if the bit is 1; otherwise stores "no".
`return labels`	Returns all 25 channel labels.

How to explain to sir:

"The Analysis file contains wake-detection information in mask format. I decode this mask into individual channel labels. Each bit represents a channel, where 1 means yes and 0 means no, according to the configured bit order."

Take your curiosity further
Scan to get the ChatGPT app

Important: The notebook assumes that the first channel corresponds to the least significant bit. This bit-to-channel mapping should be confirmed against your project's actual data specification.

Cell 5 – Converting sonar signals into spectrogram images

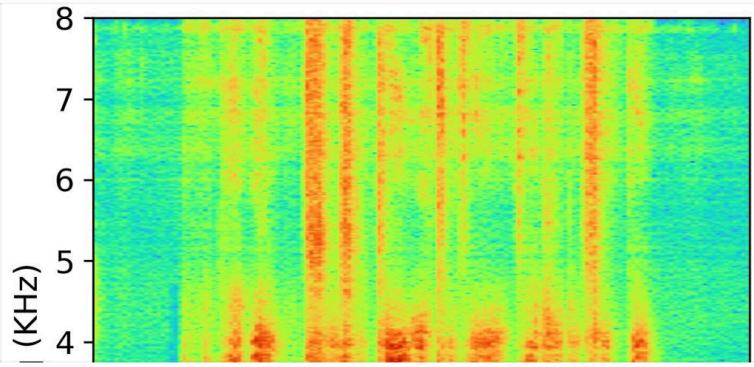
Purpose: Convert the numerical sonar signal into an image that EfficientNet can process.

Code line	Simple explanation
`IMAGE_SIZE = 96`	Sets the image height and width to 96 × 96 pixels.
`CHANNEL_COUNT = 25`	Specifies that each Beam file has 25 I channels.
`def signal_to_image(signal):`	Defines a function to convert one signal into an image.
`signal = np.asarray(signal, dtype=np.float32)`	Converts the signal into a NumPy array using decimal-number format.
`signal = np.nan_to_num(signal)`	Replaces missing or invalid numerical values with safe numerical values.
`if len(signal) < 32:`	Checks whether the signal contains fewer than 32 samples.
`raise ValueError("Signal is too short")`	Stops processing if the signal is too short.
`frequencies, times, power = spectrogram(`	Starts calculating the spectrogram.
`signal,`	Supplies the sonar signal as input.
`fs=1.0,`	Sets the sampling frequency parameter to 1.0.
`nperseg=min(128, len(signal)),`	Sets the segment length to 128 or the signal length, whichever is smaller.
`noverlap=min(96, min(128, len(signal)) - 1)`	Sets how many samples overlap between consecutive segments.
`)`	Completes the spectrogram calculation.

Take your curiosity further
Scan to get the ChatGPT app

`power = 10 * np.log10(power + 1e-10)`	Converts the power values into a logarithmic scale, avoiding a logarithm of zero.
`power = power - np.min(power)`	Shifts the smallest power value to zero.
`maximum = np.max(power)`	Finds the maximum power value.
`if maximum > 0:`	Checks whether the maximum value is greater than zero.
`power = power / maximum`	Normalizes the values between 0 and 1.
`image = tf.image.resize(`	Starts resizing the spectrogram.
`power[..., np.newaxis],`	Adds an image-channel dimension to the spectrogram.
`(IMAGE_SIZE, IMAGE_SIZE)`	Sets the required image size to 96 × 96.
`)`	Completes the resizing operation.
`.numpy()`	Converts the TensorFlow result into a NumPy array.
`image = np.repeat(image, 3, axis=-1)`	Copies the single image channel into three channels, like an RGB image.
`return (image * 255).astype(np.float32)`	Scales the image values and returns the final image as floating-point numbers.

What is a spectrogram?



A spectrogram represents a signal using colours or intensity levels to show how its frequency content changes over time.

How to explain to sir:

Take your curiosity further
Scan to get the ChatGPT app

"EfficientNet is an image-based deep-learning model. Our original input is a numerical sonar signal, so I convert each channel signal into a spectrogram image. I resize it to 96 by 96 pixels and convert it into a three-channel image so that it can be given to EfficientNet."

Cell 6 – Preparing training data

Purpose: Convert all 50 Beam files into images and prepare their corresponding labels.

Code line	Simple explanation
`X = []`	Creates an empty list to store training images.
`y = []`	Creates an empty list to store the correct training labels.
`groups = []`	Creates a list to store the PRI number associated with each sample.
`for file_path in sorted(beam_files):`	Reads the Beam files in sorted order.
`pri = get_file_pri_number(file_path)`	Extracts the PRI number from the current filename.
`print("Processing PRI:", pri)`	Displays which PRI is being processed.
`beam = pd.read_csv(file_path, header=None)`	Reads the Beam CSV without treating the first row as a header.
`beam = beam.apply(pd.to_numeric, errors="coerce")`	Converts the CSV values into numbers; invalid values become missing values.
`if beam.shape[1] != 50:`	Checks whether the Beam file contains exactly 50 columns.
`raise ValueError(...)`	Stops if the number of columns is incorrect.
`i_channels = beam.iloc[:, ::2]`	Selects columns 0, 2, 4, and so on, assuming these are the I channels.
`labels = decode_mask(analysis_map[pri], CHANNEL_COUNT)`	Gets the wake/no-wake labels for this PRI from the Analysis file.
`for channel_index in range(CHANNEL_COUNT):`	Processes all 25 I channels one by one.

Take your curiosity further
Scan to get the ChatGPT app

`signal = i_channels.iloc[:, channel_index].dropna().values`	Extracts the current channel's signal and removes missing values.
`image = signal_to_image(signal)`	Converts the extracted signal into a spectrogram image.
`X.append(image)`	Adds that image to the training images.
`y.append(1 if labels[channel_index] == "yes" else 0)`	Converts the channel label into a number: 1 for yes and 0 for no.
`groups.append(pri)`	Stores the PRI number for this training sample.
`X = np.array(X, dtype=np.float32)`	Converts the image list into a NumPy array.
`y = np.array(y, dtype=np.float32)`	Converts the label list into a NumPy array.
`groups = np.array(groups)`	Converts the PRI group list into a NumPy array.
`print("Training images:", X.shape)`	Displays the size of the training-image dataset.
`print("Labels:", y.shape)`	Displays the number of labels.
`print("PRI groups:", len(np.unique(groups)))`	Displays the number of unique PRI groups.
`print("Yes labels:", int(np.sum(y == 1)))`	Counts how many positive labels exist.
`print("No labels:", int(np.sum(y == 0)))`	Counts how many negative labels exist.

How to explain to sir:

"In this step, the program processes all 50 Beam files. Each file contains 25 I channels. I convert each channel into a spectrogram image and assign its corresponding label from the Analysis file. Finally, I prepare the image dataset X and label dataset y for model training."

For example, if all 50 files contain 25 valid I channels, the dataset will contain:

- 50 × 25 = 1,250 training images.
- 1,250 corresponding labels.

Cell 7 – Building the EfficientNetB0 model

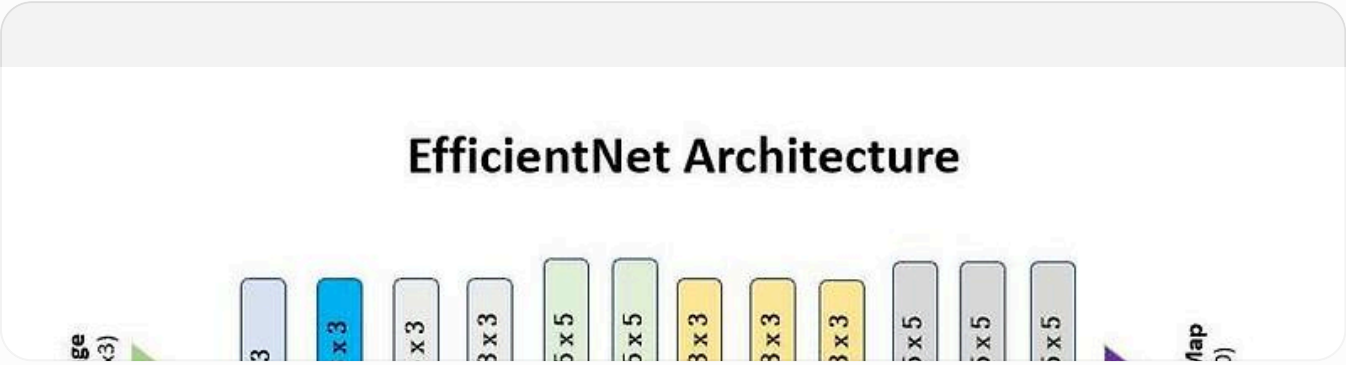
Purpose: Create the deep-learning model that learns to classify wake and non-wake signals.

Take your curiosity further
Scan to get the ChatGPT app

Code line	Simple explanation
`tf.keras.utils.set_random_seed(42)`	Sets a fixed random seed to help make results more repeatable.
`inputs = Input(shape=(IMAGE_SIZE, IMAGE_SIZE, 3))`	Defines the input image size as 96 × 96 with 3 channels.
`base_model = EfficientNetB0(`	Starts creating the EfficientNetB0 model.
`weights=None,`	Initializes the model without pretrained ImageNet weights. It starts with randomly initialized weights.
`include_top=False,`	Removes EfficientNet's original classification output layer.
`input_tensor=inputs`	Connects the defined input images to EfficientNet.
`)`	Completes the EfficientNet model creation.
`x = GlobalAveragePooling2D()(base_model.output)`	Reduces the feature maps into a compact feature vector.
`x = Dropout(0.3)(x)`	Randomly drops 30% of units during training to help reduce overfitting.
`outputs = Dense(1, activation="sigmoid")(x)`	Creates one output value between 0 and 1 for binary classification.
`model = Model(inputs, outputs)`	Combines the input and output layers into one complete model.
`model.compile(`	Configures how the model will learn.
`optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),`	Uses the Adam optimizer with a learning rate of 0.001.
`loss="binary_crossentropy",`	Uses binary cross-entropy as the loss function for yes/no classification.
`metrics=["accuracy"]`	Tracks the accuracy during training.
`)`	Completes model configuration.
`model.summary()`	Displays the model architecture and its trainable parameters.

Take your curiosity further
Scan to get the ChatGPT app

What is EfficientNetB0?



EfficientNetB0 is a convolutional neural network (CNN). It learns patterns and features from images.

How to explain to sir:

"I selected EfficientNetB0 as the deep-learning architecture. The model extracts features from the spectrogram images. I added a global average pooling layer, dropout layer and a sigmoid output layer to perform binary classification into wake and non-wake."

Cell 8 – Training the model

Purpose: Train the EfficientNet model using the prepared images and labels.

Code line	Simple explanation
`history = model.fit(`	Starts training the model and stores the training history.
`X,`	Supplies the spectrogram images as training input.
`y,`	Supplies the correct yes/no labels.
`epochs=3,`	Trains the model for 3 complete passes through the training dataset.
`batch_size=8,`	Processes 8 images in each training batch.
`shuffle=True,`	Shuffles the training samples during training.
`verbose=1`	Displays the training progress.
`)`	Completes the training operation.

Take your curiosity further
Scan to get the ChatGPT app

What is an epoch?

One epoch means the model has gone through the entire training dataset once.

What is batch size?

Batch size 8 means the model processes 8 images at a time before updating its weights.

How to explain to sir:

"After preparing the dataset, I trained the EfficientNetB0 model using the images and their labels. I used 3 epochs and a batch size of 8. During training, the model learns patterns that help distinguish wake signals from non-wake signals."

Cell 9 – Saving the trained model

Purpose: Save the trained model so that it can be reused without training again.

Code line	Simple explanation
`MODEL_PATH = os.path.join(MODEL_DIR, "wake_efficientnet.keras")`	Creates the complete path and filename for saving the model.
`model.save(MODEL_PATH)`	Saves the trained model in Keras format.
`print("Model saved:", MODEL_PATH)`	Displays the location where the model was saved.

How to explain to sir:

"Once training is completed, I save the model as `wake_efficientnet.keras`. This saved model can be loaded later for prediction, so we do not need to train it every time."

Cell 10 – Predicting wake detection and generating output CSV files

Purpose: Use the trained model to predict the labels for all 50 Beam files.

This is the most important cell for demonstrating the actual use of your trained model.

Code line	Simple explanation

Take your curiosity further
Scan to get the ChatGPT app

`for file_path in sorted(beam_files):`	Goes through all 50 Beam files one by one.
`pri = get_file_pri_number(file_path)`	Extracts the PRI number from the filename.
`print("Predicting PRI:", pri)`	Displays which PRI is currently being predicted.
`beam = pd.read_csv(file_path, header=None)`	Reads the Beam CSV file without a header.
`beam = beam.apply(pd.to_numeric, errors="coerce")`	Converts the signal values into numbers.
`i_channels = beam.iloc[:, ::2]`	Selects the I-channel columns.
`images = []`	Creates an empty list to store the images for prediction.
`for channel_index in range(CHANNEL_COUNT):`	Loops through the 25 I channels.
`signal = i_channels.iloc[:, channel_index].dropna().values`	Extracts the current channel's signal and removes missing values.
`image = signal_to_image(signal)`	Converts the signal into a spectrogram image using the same preprocessing function used during training.
`images.append(image)`	Adds the image to the prediction list.
`images = np.array(images, dtype=np.float32)`	Converts all 25 images into a NumPy array.
`probabilities = model.predict(`	Starts the model prediction process.
`images,`	Supplies the 25 images to the trained model.
`verbose=0`	Hides the detailed prediction progress.
`).flatten()`	Converts the prediction output into a one-dimensional array.
`predicted_labels = [`	Starts creating the final yes/no labels.
`"yes" if probability >= 0.5 else "no"`	If the predicted probability is at least 0.5, the label is yes; otherwise it is no.
`for probability in probabilities`	Applies this condition to every predicted probability.

Take your curiosity further
Scan to get the ChatGPT app

`]`	Completes the predicted-label list.
`result = pd.DataFrame({`	Creates a Pandas DataFrame for the output.
`"Channel": [`	Defines the Channel column.
`f"Channel_{i+1}" for i in range(CHANNEL_COUNT)`	Generates Channel_1 through Channel_25.
`],`	Completes the Channel column.
`"Label": predicted_labels`	Adds the predicted yes/no labels.
`})`	Completes the output DataFrame.
`output_path = os.path.join(`	Starts creating the output file path.
`OUTPUT_DIR,`	Specifies the Output folder.
`f"Wake_Labeled_PRI_{pri}.csv"`	Creates a filename using the current PRI number.
`)`	Completes the output path.
`result.to_csv(output_path, index=False)`	Saves the output as a CSV file without an extra index column.
`print("All output files generated successfully!")`	Displays a message after all files have been processed.

Example output

ILLUSTRATIVE EXAMPLE – NOT ACTUAL MODEL PREDICTIONS

Channel	Label
Channel_1	no
Channel_2	yes
Channel_3	no
Channel_4	yes
Channel_5	no

Take your curiosity further
Scan to get the ChatGPT app

...	...
Channel_25	yes

The program generates separate files such as:

- `Wake_Labeled_PRI_1.csv`
- `Wake_Labeled_PRI_2.csv`
- `Wake_Labeled_PRI_3.csv`
- ...
- `Wake_Labeled_PRI_50.csv`

How to explain to sir:

"This is the prediction stage. After training, the saved model is used to predict the 25 channels in each Beam file. The model gives a probability for each channel. Based on a 0.5 threshold, the program classifies each channel as yes or no. Finally, it saves one CSV output file for every PRI."

Cell 11 – Checking the generated output files

Purpose: Verify that the output CSV files have been generated correctly.

Code line	Simple explanation
`output_files = glob.glob(`	Searches for generated output files.
`os.path.join(OUTPUT_DIR, "Wake_Labeled_PRI_*.csv")`	Finds all CSV files whose names begin with `Wake_Labeled_PRI_`.
`)`	Completes the file search.
`print("Total output files:", len(output_files))`	Displays the total number of generated files.
`for path in sorted(output_files):`	Goes through each output file in sorted order.
`df = pd.read_csv(path)`	Reads the current output CSV file.
`print(os.path.basename(path), df.shape)`	Displays its filename and dimensions.
`print(df.head(3))`	Displays the first three rows for checking.

Take your curiosity further
Scan to get the ChatGPT app

How to explain to sir:

"In the final step, I verify the generated files. I check the total number of output CSVs, their dimensions and sample records to make sure the prediction results have been saved correctly."

Cell 12 – Empty cell

This cell does not contain any code. It has no effect on the project.

Important questions your sir may ask

Here are some likely questions and simple answers you can give.

1. Why did you use EfficientNetB0?

"EfficientNetB0 is a CNN-based deep-learning architecture. I use it to learn features from spectrogram images and perform wake/no-wake classification."

2. Why convert sonar signals into images?

"The original data is numerical signal data. Converting it into spectrogram images allows an image-based CNN model to learn time-frequency patterns."

3. What is the purpose of the Analysis CSV?

"It provides the wake-detection information that is decoded into labels for training the model."

4. What is the purpose of the `.keras` file?

"It stores the trained model. We can load it later and perform prediction without training the model again."

5. What is the difference between training and prediction?

Training	Prediction
Uses input images and their known labels.	Uses new input images.
Model learns patterns.	Model applies the learned patterns.

Take your curiosity further
Scan to get the ChatGPT app

Updates model weights.	Does not normally update model weights.
Saves the trained model.	Generates yes/no output labels.

6. Why do you use a 0.5 threshold?

"The sigmoid output gives a value between 0 and 1. I use 0.5 as the decision threshold: values at least 0.5 are classified as yes, and lower values as no."

Important points to know before presenting this notebook

I noticed a few technical points in your actual code. It is better to understand them before explaining the project to your sir.

1. **The model is initialized with** `weights=None`**.** This means it starts with random weights rather than pretrained ImageNet weights. It learns from your dataset from scratch.

2. **The model is trained for only 3 epochs.** This is the setting in your current notebook, not proof that the model has reached good accuracy.

3. **There is no validation or test dataset in the current training code.** The notebook trains on `X` and `y` and does not independently measure performance on unseen data.

4. **The** `groups` **list is created but not used for splitting.** The notebook imports `GroupShuffleSplit`, but it does not actually perform a group-based train/test split.

5. **The Analysis CSV labels are used to train the model.** The model's predictions are generated from Beam signals, but the correctness of those predictions must be evaluated against independent, trusted labels.

6. **The bit-to-channel mapping must be verified.** The notebook assumes `lsb_first`, meaning bit 0 represents Channel 1. Confirm this with your project data specification.

These points matter because your sir may ask how you know the trained model is actually detecting wake signals correctly. You should not claim high accuracy or verified performance unless you have tested it on separate labelled data.

Take your curiosity further
Scan to get the ChatGPT app

Your short project presentation speech

You can use this when explaining the notebook from beginning to end:

Writing

Project Explanation – Sonar Wake Detection Using EfficientNetB0

Good morning, sir.

My project is based on sonar wake detection using a deep-learning approach called EfficientNetB0.

First, I collect 50 Beam PRI CSV files and one PRI Analysis CSV file. The Beam files contain the sonar signal data, and the Analysis file provides the wake-detection information.

Next, I match the PRI numbers from both files. I extract the 25 I-channel signals from each Beam file and convert each signal into a spectrogram image.

After preprocessing, I prepare the training images and their corresponding yes/no labels. I then build the EfficientNetB0 model and train it using these images and labels.

Once training is completed, I save the trained model in `.keras` format so that it can be reused without training again.

Finally, I use the trained model to predict the wake-detection result for each of the 25 channels in every Beam file. The program generates 50 separate CSV files containing the Channel and Label columns.

The main objective is to automate sonar wake detection using a trained deep-learning model and generate the prediction results in CSV format.

Thank you, sir.

One important distinction for your demonstration: This notebook currently contains both training and prediction code. Your final GUI should load the saved `wake_efficientnet.keras` model and run the prediction part automatically when you select the input files. It should not train the model again every time you use the GUI.

Take your curiosity further
Scan to get the ChatGPT app

You

where are spectrograms how to find and get them as far i havent seen them

ChatGPT

You haven't missed anything. **Your current notebook generates spectrograms internally, but it does not display them on the screen or save them as image files.** That is why you haven't seen any spectrogram images in your folders.

In your notebook, spectrograms are created in **Cell 5**, inside this function:

```
def signal_to_image(signal):
    ...
    frequencies, times, power = spectrogram(...)
    ...
    return (image * 255).astype(np.float32)
```

The important line is:

```
frequencies, times, power = spectrogram(...)
```

This calculates the spectrogram. The later lines resize it and convert it into an image for EfficientNet. But there is no `plt.show()` or `plt.savefig()` command, so you don't see or retain the images.

How to get and see the spectrograms

You can add a small section to your notebook to save the spectrogram images as PNG files.

Step 1: Add Matplotlib import

In Cell 1, add this line:

Take your curiosity further
Scan to get the ChatGPT app

```
import matplotlib.pyplot as plt
```

Step 2: Add a folder for spectrogram images

In Cell 2, below the `MODEL_DIR` line, add:

```
SPECTROGRAM_DIR = os.path.join(BASE_DIR, "Spectrograms")
os.makedirs(SPECTROGRAM_DIR, exist_ok=True)
```

This creates a new folder:

```
D:\Wake_EfficientNet\Spectrograms
```

Step 3: Add code to display and save spectrograms

Add a new cell **after Cell 6** and before Cell 7.

Copy this complete code:

```
# Display and save sample spectrograms

import matplotlib.pyplot as plt

# Select the first Beam CSV file
sample_file = sorted(beam_files)[0]

# Read the Beam data
beam = pd.read_csv(sample_file, header=None)
beam = beam.apply(pd.to_numeric, errors="coerce")

# Select the first I channel
signal = beam.iloc[:, 0].dropna().values

# Generate spectrogram values
frequencies, times, power = spectrogram(
```



Take your curiosity further

Scan to get the ChatGPT app



```
    signal,
    fs=1.0,
    nperseg=min(128, len(signal)),
    noverlap=min(96, min(128, len(signal)) - 1)
)

# Convert power into decibels
power_db = 10 * np.log10(power + 1e-10)

# Display the spectrogram
plt.figure(figsize=(10, 5))
plt.pcolormesh(times, frequencies, power_db, shading="auto")
plt.colorbar(label="Power (dB)")
plt.xlabel("Time")
plt.ylabel("Frequency")
plt.title("Sonar Spectrogram - First Beam PRI, Channel 1")
plt.tight_layout()

# Save the image
plt.savefig(
    os.path.join(SPECTROGRAM_DIR, "PRI_1_Channel_1.png"),
    dpi=150
)

# Show the image in Jupyter
plt.show()

print("Spectrogram saved successfully!")
```

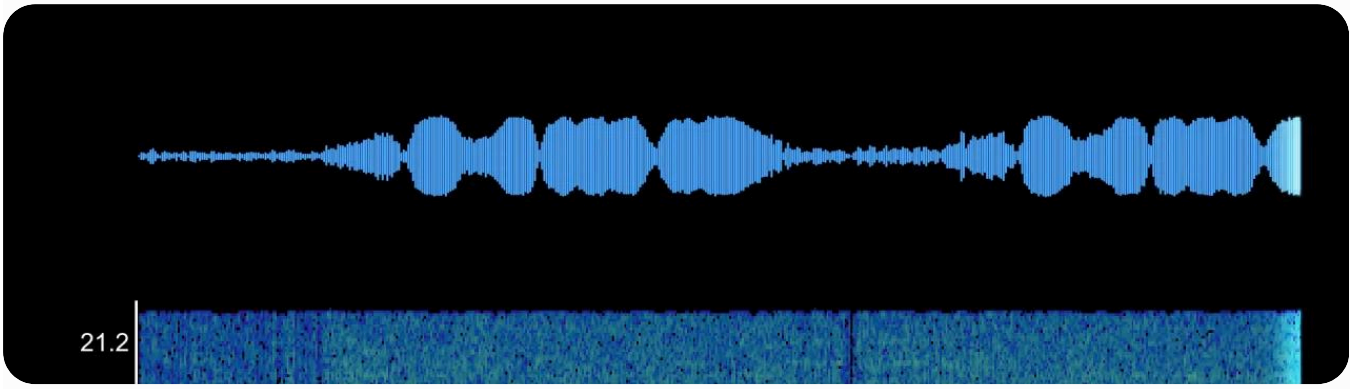
Step 4: Run the new cell

1. Open your Jupyter Notebook.
2. Run Cell 1 and Cell 2 first.
3. Run the new spectrogram cell.
4. You should see a coloured spectrogram plot inside Jupyter.
5. Open your Windows File Explorer and go to:

`D:\Wake_EfficientNet\Spectrograms`

Take your curiosity further
Scan to get the ChatGPT app

You should find `PRI_1_Channel_1.png` there.



Step 5: If you want all 50 × 25 spectrogram images

Use this additional code in a new cell:

```
for file_path in sorted(beam_files):

    pri = get_file_pri_number(file_path)

    beam = pd.read_csv(file_path, header=None)
    beam = beam.apply(pd.to_numeric, errors="coerce")

    i_channels = beam.iloc[:, ::2]

    for channel_index in range(CHANNEL_COUNT):

        signal = i_channels.iloc[:, channel_index].dropna().values

        image = signal_to_image(signal)

        filename = f"PRI_{pri}_Channel_{channel_index+1}.png"

        plt.imsave(
            os.path.join(SPECTROGRAM_DIR, filename),
            image.astype(np.uint8)
        )

print("All spectrogram images saved!")
```

Take your curiosity further
Scan to get the ChatGPT app

Expected result

50 files × 25 channels = **1,250 spectrogram images**, assuming all channels are valid.

For your sir's demonstration: First show one spectrogram image from Step 3. Explain that the same conversion is applied to all 25 channels in every Beam file. Then show Cell 7, where EfficientNet receives these images for training.

One distinction: the images created in Step 5 use the same preprocessing as the model. They are saved for inspection; your original training code does not need to save them to train the model.

For the underlying spectrogram calculation, see the SciPy spectrogram documentation ↗.

Take your curiosity further
Scan to get the ChatGPT app